Use JavaScript for doing client – side validation of the pages

implemented in experiment 1 andexperiment 2.

```
const top_items_count = document.querySelector('.items-count'),
bottom_items_count = document.querySelector('.bottom-items-count'),
exit = document.querySelector('.exit'),
bucket = document.querySelector('.cart').style;
var items = document.querySelectorAll('.items');
top_items_count.innerHTML = count;
bottom_items_count.innerHTML = count;
function openBucket() {
bucket.visibility = "visible";
bucket.opacity = "1";
bucket.zIndex = "9";
bucket.transition = "all 0.5s";
}
exit.addEventListener('click', () => {
bucket.visibility = "hidden";
bucket.opacity = "0";
bucket.zIndex = "-9";
bucket.transition = "all 0.5s";
});
var food_cart = [];
() => {
if (localStorage.food_cart) {
food_cart = JSON.parse(localStorage.food_cart);
showCart();
}
}
```

```
var qty=1;
for (i = 0; i <= items.length - 1; i++) {
var count=0;
items[i].onclick = e => {
count=count+1
var itemName = e.target.dataset.item;
var price = e.target.dataset.price;
addToCart(itemName, price, qty);
top_items_count.innerHTML = count;
bottom_items_count.innerHTML = count;
}
}
function addToCart(itemName, price,qty) {
for (var i in food_cart) {
if (food_cart[i].Product == itemName) {
food_cart[i].Qty += qty;
showCart();
saveCart();
return;
}
}
var itemArray = {
Product: itemName,
Price: price,
Qty: qty
}
food_cart.push(itemArray)
saveCart();
showCart();
```

```
}
function saveCart() {
if (window.localStorage) {
localStorage.food_cart = JSON.stringify(food_cart);
}
}
function deleteItem(index) {
food_cart.splice(index, 1);
showCart();
saveCart();
}
function showCart() {
if (food_cart.length == 0) {
var _ul = document.querySelector('#ul');
_ul.innerHTML = "";
return;
}
var _ul = document.querySelector('#ul');
_ul.innerHTML = "";
for (var i in food_cart) {
var item = food_cart[i];
var li = document.createElement("li")
var row = `<span>${item.Product}</span><span onclick='deleteItem(" + i + ")'><i class='fas
fa-trash'></i></span><span>${item.Qty}</span><span>${item.Qty * item.Price}</span>`;
li.innerHTML += row;
var _ul = document.querySelector('#ul');
_ul.appendChild(li);
}
}
```

Create a custom server using http module and explore the other modules of Node JS

like OS,path, event.

The http.createServer() method includes request and response parameters which is supplied by Node.js. The request object can be used to get information about the current HTTP request e.g., url, request header, and data. The response object can be used to send a response for a current HTTP request.

The following example demonstrates handling HTTP request and response in Node.js.

```
var http = require('http'); // Import Node.js core module
var server = http.createServer(function (req, res) { //create web
server
if (req.url == '/') { //check the URL of the current request
// set response header
res.writeHead(200, { 'Content-Type': 'text/html' });
// set response content
res.write('<html><body><p>This is home
Page.</p></body></html>');
res.end();
}
elseif (req.url == "/student") {
res.writeHead(200, { 'Content-Type': 'text/html' });
res.write('<html><body><p>This is student
Page.</p></body></html>');
res.end();
}
elseif (req.url == "/admin") {
res.writeHead(200, { 'Content-Type': 'text/html' });
res.write('<html><body><p>This is admin
```

```
Page.</p></body></html>');
res.end();
}
else
res.end('Invalid Request!');
});
server.listen(5000); //6 - listen for any incoming requests
console.log('Node.js web server at port 5000 is running..')
```

Now, run the above web server as shown below.

```
C:\> node server.js
Node.js web server at port 5000 is running..
```

To test it, you can use the command-line program curl, which most Mac and Linux machines have pre-installed.

```
curl -i http://localhost:5000
```

You should see the following response.

```
HTTP/1.1 200 OK
Content-Type: text/plain
Date: Tue, 8 Sep 2015 03:05:08 GMT
Connection: keep-alive
This is home page.
```

For Windows users, point your browser to http://localhost:5000 and see the following result.